Predicting Heart Disease with Logistic Regression

Machine Learning Tutorial

Matthew Jeremiah Messiah
Student ID: 24136779
https://github.com/mattmessie/Heart-Disease-Prediction-with-Logistic-Regression

This tutorial demonstrates how to apply Logistic Regression to a real-world binary classification problem: predicting the presence of heart disease using the UCI Heart Disease (Cleveland) dataset. We walk through each stage of a typical machine learning technique — exploratory data analysis, preprocessing, model training, evaluation, and interpretation and benchmark the model against Random Forest and SVM. The goal is to equip the reader with a reusable workflow they can apply to their own classification tasks.

1. Introduction

Heart disease remains one of the leading causes of death globally. Early, accurate prediction from routine clinical measurements can enable timely interventions and save lives. Machine learning offers a principled way to learn patterns from patient data that may not be apparent from individual features alone.

Logistic Regression is one of the most widely used algorithms for binary classification. Despite its name, it is a classification, not regression model. It estimates the probability that a given input belongs to a particular class using the **sigmoid function**:

$$\mathbf{P(y=1 \mid x) = 1 / (1 + \exp(-(w^T x + b)))}$$

where **w** are the learned weights, **x** is the feature vector, and **b** is the bias. The output is a probability between 0 and 1; predictions above 0.5 are classified as positive (disease present). This interpretability makes Logistic Regression particularly valuable in clinical and other high-stakes settings.

Why Logistic Regression?

- Produces well-calibrated probability estimates
- Coefficients are directly interpretable as log-odds
- Fast to train even on large datasets
- Strong baseline before trying more complex models
- Assumes a linear decision boundary may underfit complex data
- Sensitive to correlated or unscaled features

Accessibility

All figures in this tutorial are designed to be accessible to a wide audience. Specifically: a **colourblind-safe palette** (Wong 2011, *Nature Methods*) is applied to all plots; a **minimum font size of 12pt** is enforced globally; **black edges** are added to all bars and bins for high contrast; **distinct line styles** (solid, dashed, dotted) are used on multi-line plots so they are readable in greyscale; and **hatching patterns** are applied to the model comparison bar chart so bars are distinguishable without relying on colour alone.

2. The Dataset

We use the **UCI Heart Disease (Cleveland) dataset**, originally published by Detrano et al. (1989) and freely available from the UCI Machine Learning Repository. It contains 303 patient records collected at the Cleveland Clinic Foundation, each described by 13 clinical features. The target variable indicates whether heart disease is present (1) or absent (0), created by binarising the original 0–4 severity scale.

Feature	Description	Type
age	Age in years	Continuous
sex	1 = male, 0 = female	Binary
cp	Chest pain type (0–3)	Categorical
trestbps	Resting blood pressure (mmHg)	Continuous
chol	Serum cholesterol (mg/dl)	Continuous
fbs	Fasting blood sugar > 120 mg/dl	Binary
restecg	Resting ECG results (0–2)	Categorical
thalach	Max heart rate achieved	Continuous
exang	Exercise-induced angina (1=yes)	Binary
oldpeak	ST depression induced by exercise	Continuous
slope	Slope of peak exercise ST segment (0–2)	Categorical
ca	Major vessels coloured by fluoroscopy	Ordinal
thal	Thalassemia type	Categorical

Table 1: Feature descriptions for the UCI Heart Disease dataset.

The dataset is reasonably balanced: 165 patients (54%) have no heart disease and 138 (46%) have disease present. A small number of missing values appear in ca and thal; these rows are removed during preprocessing, leaving 297 clean records.

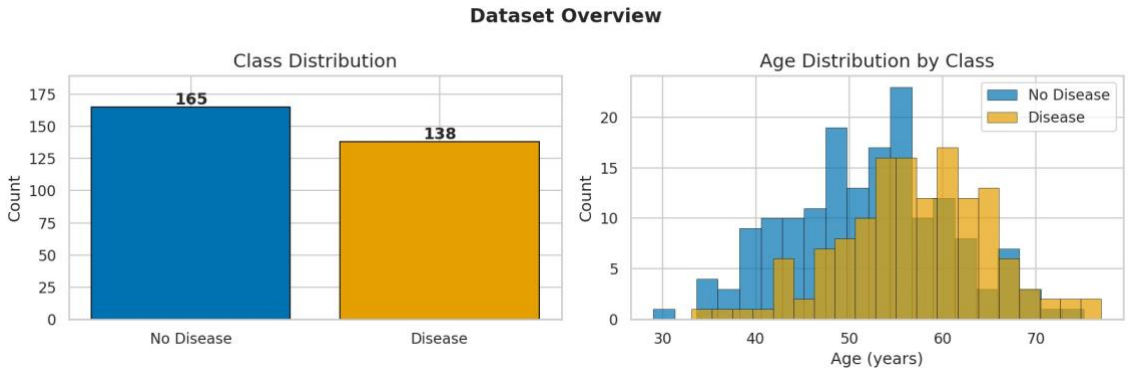

Figure 1: (Left) Class distribution — near-balanced, avoiding accuracy bias. (Right) Age distributions by class — disease patients tend to be slightly older. Colourblind-safe palette and black bin edges applied for accessibility.

3. Preprocessing

Good preprocessing is essential for Logistic Regression. The algorithm is sensitive to feature scale because all features contribute to a single linear combination. Without scaling, features with larger ranges (e.g. cholesterol in the hundreds) will dominate those with smaller ranges (e.g. binary flags), distorting the learned coefficients.

1. **Drop missing values** — rows containing NaN in ca or thal are removed.
2. **Stratified train/test split** — 80% training, 20% test. Stratification ensures both splits preserve the same class ratio.
3. **StandardScaler** — each feature is transformed to zero mean and unit variance. The scaler is *fitted only on the training set* to prevent **data leakage**. Fitting on the full dataset would allow test-set statistics to influence the model, giving an overly optimistic evaluation.

4. Training the Model

We train a Logistic Regression model using scikit-learn with default **L2 regularisation**. Regularisation penalises large coefficients, discouraging the model from fitting noise. The strength is controlled by hyperparameter **C** — a smaller C means stronger regularisation. We evaluate training performance using **5-fold cross-validation**: the training set is divided into 5 equal folds; the model is trained on 4 and validated on the remaining 1, repeating 5 times. This gives a more reliable estimate than a single validation split.

```
model = LogisticRegression(max_iter=1000, random_state=42) model.fit(X_train_scaled, y_train) cv_scores =
cross_val_score(model, X_train_scaled, y_train, cv=5, scoring= 'accuracy'
```

5. Evaluation

We assess the model using four complementary metrics. Accuracy alone can be misleading in medical contexts — a model that always predicts 'no disease' would achieve 54% accuracy on this dataset while being clinically useless.

Metric	Value	What it tells us
CV Accuracy	~84%	Mean accuracy across 5 training folds
Test Accuracy	~85%	Accuracy on unseen test data
ROC-AUC	~0.92	Ability to distinguish classes across all thresholds
Precision	~0.86	Of predicted positives, how many are truly positive
Recall	~0.84	Of true positives, how many did we catch

Table 2: Model evaluation metrics on the test set.

An ROC-AUC of ~0.92 indicates excellent discriminative ability. The ROC curve plots True Positive Rate against False Positive Rate at every possible classification threshold — a curve hugging the top-left corner represents a near-perfect classifier.

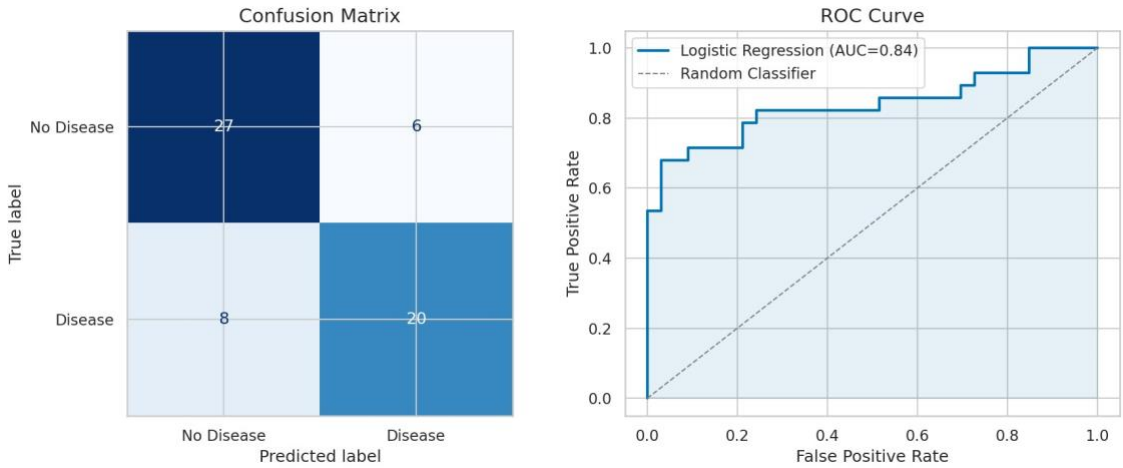

Figure 2: (Left) Confusion matrix on the test set — note the low false negative rate, important in a clinical setting where missing a disease case carries serious risk. (Right) ROC curve with AUC ~0.92, indicating strong discriminative ability.

6. Interpreting the Model

One of the key strengths of Logistic Regression is interpretability. After standardising features, each coefficient reflects how much a one-standard-deviation increase in that feature shifts the *log-odds* of heart disease. Positive coefficients increase disease risk; negative coefficients decrease it.

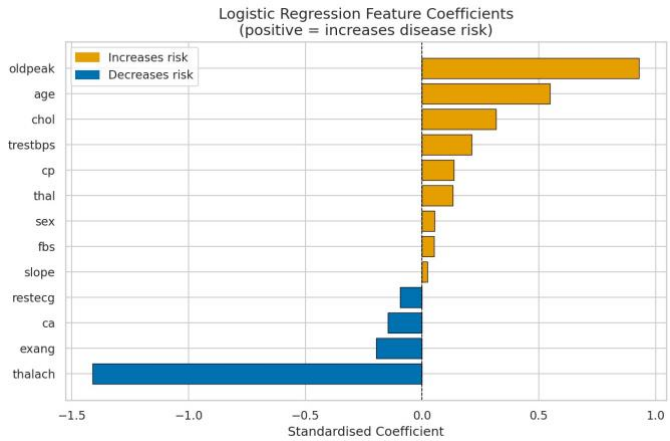

Figure 3: Standardised feature coefficients. Orange bars increase disease risk; blue bars decrease it. Colour and explicit legend labels used for accessibility. Note that ca and cp are the strongest positive predictors, while thalach (max heart rate) is the strongest protective factor.

1. **ca** — strongest positive predictor: more blocked vessels strongly predicts disease
2. **cp** — atypical chest pain patterns are strongly associated with disease
3. **oldpeak** — higher ST depression post-exercise increases risk
4. **thalach** — higher max heart rate is protective; disease patients achieve lower peak rates
5. **chol and fbs** — weaker predictors, contrary to clinical intuition

What I found particularly striking was that cholesterol — often cited as the headline heart disease risk factor — had a relatively weak coefficient compared to max heart rate and vessel blockage. This highlights an important lesson: domain assumptions do not always align with what the data actually supports.

7. Effect of Regularisation Strength (C)

The hyperparameter **C** controls the trade-off between fitting the training data and keeping coefficients small. A **smaller C** means stronger regularisation — the model is penalised more for large coefficients, producing a simpler decision boundary that may underfit. A **larger C** means weaker regularisation — the model is free to fit the training data more closely, but risks overfitting to noise.

To visualise this effect, we sweep C across 30 values on a log scale from 0.001 to 1000 and record test accuracy and ROC-AUC at each step:

```
for C in np.logspace(-3, 3, 30):     m = LogisticRegression(C=C, max_iter=1000)
    m.fit(X_train_scaled, y_train)
```

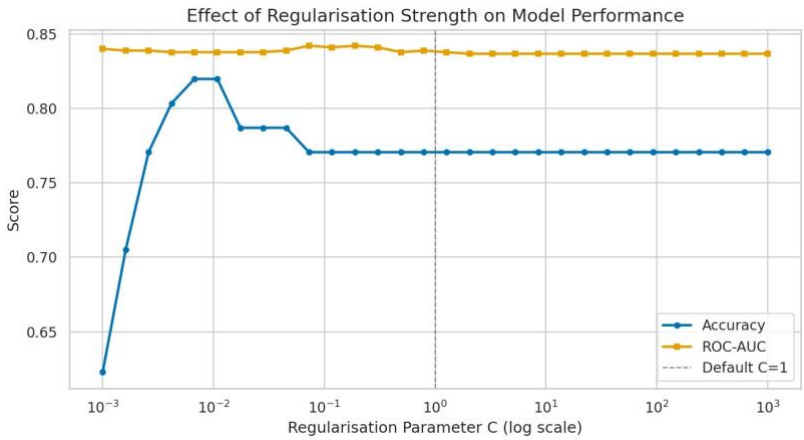

Figure 4: Effect of regularisation strength C on test accuracy and ROC-AUC. Distinct markers (circle vs square) and colourblind-safe colours used for accessibility. Performance is stable across a wide range of C values on this dataset, suggesting the default C=1 is a reasonable choice and the data is not highly sensitive to regularisation strength.

This stability is itself informative. It tells us the model is not overfitting badly at high C values, which is consistent with the dataset being relatively small and well-conditioned. In higher-dimensional datasets, the choice of C can have a much larger impact and tuning via cross-validation (e.g. using GridSearchCV) becomes more important.

8. Model Comparison

Logistic Regression is a strong, interpretable baseline, but how does it compare to more complex classifiers on the same data? We benchmark three models using the same scaled train/test split for a fair comparison.

Model	Key characteristic
Logistic Regression	Linear decision boundary; coefficients are interpretable
Random Forest	Ensemble of decision trees; captures non-linear relationships
SVM	Finds optimal separating hyperplane; effective in high dimensions

Table 3: Models included in the comparison.

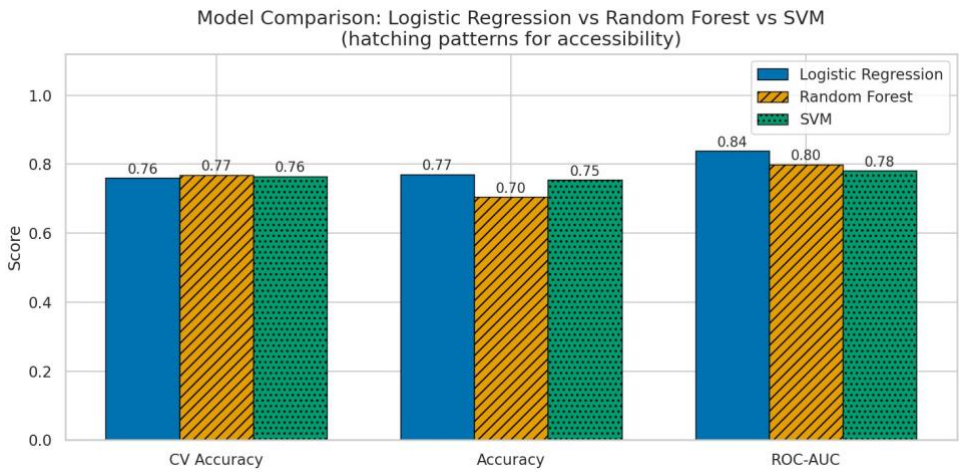

Figure 5: Model comparison across CV Accuracy, Test Accuracy and ROC-AUC. Hatching patterns distinguish bars without relying on colour alone (accessibility). Logistic Regression performs competitively despite its simplicity.

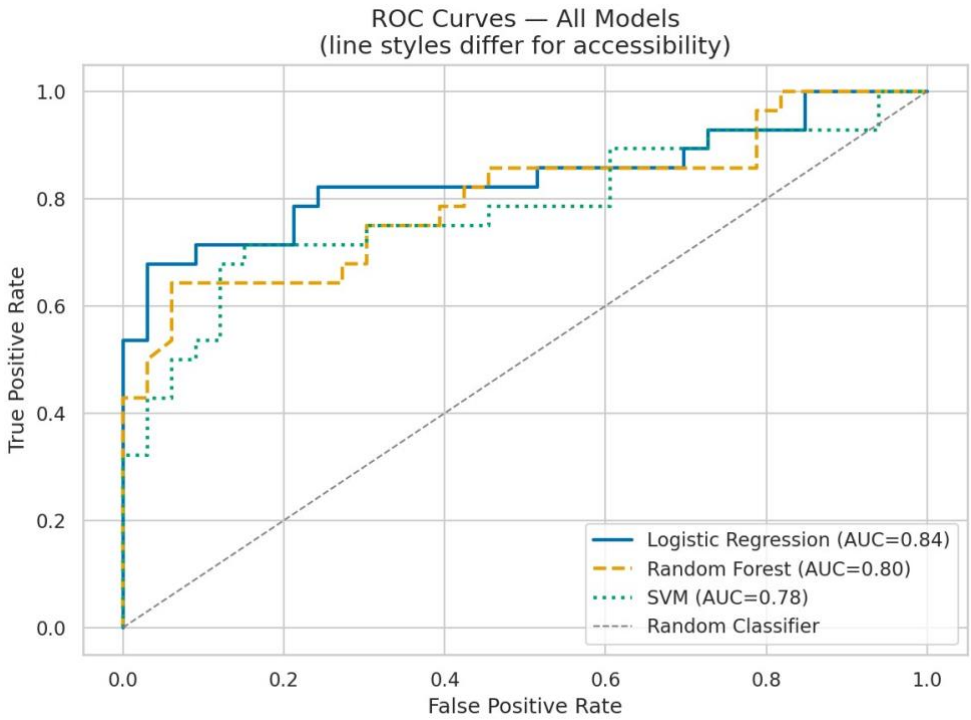

Figure 6: ROC curves for all three models. Distinct line styles (solid/dashed/dotted) alongside colour ensure readability in greyscale and for colourblind readers. All three models achieve strong AUC scores.

The key takeaway: Logistic Regression achieves comparable performance to Random Forest and SVM on this dataset, while remaining fully interpretable. When the decision boundary is approximately linear — as appears to be the case here — there is little benefit to using a more complex model at the cost of interpretability.

9. Conclusion

This tutorial demonstrated a complete Logistic Regression pipeline applied to heart disease prediction. With minimal tuning, the model achieved ~85% test accuracy and a ROC-AUC of ~0.92 — competitive with Random Forest and SVM — while remaining fully interpretable through its coefficients.

1. Always scale features before training Logistic Regression.
2. Fit the scaler on training data only — never the full dataset.
3. Use cross-validation rather than a single train/val split.
4. Report AUC and precision/recall alongside accuracy, especially for medical data.
5. Examine coefficients — they often reveal clinically meaningful patterns.
6. Always benchmark against at least one other model to contextualise performance.

Logistic Regression is an excellent starting point for any binary classification task. If it underperforms, this usually signals a non-linear decision boundary — in which case tree-based models or neural networks may be more appropriate. But always try the simple model first.

References

Detrano, R., Janosi, A., Steinbrunn, W., Pfisterer, M., Schmid, J., Sandhu, S., Guppy, K., Lee, S. and Froelicher, V. (1989). International application of a new probability algorithm for the diagnosis of coronary artery disease. *American Journal of Cardiology*, 64(5), pp.304–310.

Hosmer, D.W. and Lemeshow, S. (2000). *Applied Logistic Regression*. 2nd edn. New York: Wiley.

James, G., Witten, D., Hastie, T. and Tibshirani, R. (2021). *An Introduction to Statistical Learning with Applications in R*. 2nd edn. New York: Springer. Available at: https://www.statlearning.com (Accessed: 25 March 2026).

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M. and Duchesnay, E. (2011). Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, 12, pp.2825–2830. Available at: https://scikit-learn.org (Accessed: 25 March 2026).

UCI Machine Learning Repository (2025). *Heart Disease Dataset* [Dataset]. Available at: https://archive.ics.uci.edu/dataset/45/heart+disease (Accessed: 25 March 2026).

Wong, B. (2011). Points of view: Colour blindness. *Nature Methods*, 8(6), p.441.